Unit: 5

Arrays and Strings

Normally, an array is a collection of similar type of elements that have a contiguous memory location. Java array is an object which contains elements of a similar data type. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array. Array in java is index-based, the first element of the array is stored at the 0 index.

Types of Array in java:

There are two types of array.

- Single Dimensional Array .
- Multidimensional Array.

Advantages

- Stores multiple values using one variable.
- Easy to process data using loops.
- Reduces code complexity.
- Faster access using index numbers.

Creating an Array:

Step 1: Declaration

```
int marks[];
```

OR

```
int[] marks;
```

Step 2: Memory Allocation

```
marks = new int[5];
```

Complete Example

```
public class ArrayDemo {
   public static void main(String[] args) {
```

```
        int marks[] = new int[5];

        marks[0] = 80;
        marks[1] = 85;
        marks[2] = 90;
        marks[3] = 75;
        marks[4] = 88;

        System.out.println(marks[0]);
    }
}
```

Array Initialization :

Method 1: Using new Keyword

```
int num[] = new int[5];
```

Method 2: Direct Initialization

```
int num[] = {10, 20, 30, 40, 50};
```

Array Index

Each element has an index number starting from 0.

```
int num[] = {10,20,30,40,50};
```

Array Length

Use **.length**

```
int[] arr = {1,2,3,4};

System.out.println(arr.length);   Output: 4
```

Traversing Arrays Using for loop :

```
int[] arr = {10,20,30};


for(int i = 0; i < arr.length; i++) {
   System.out.println(arr[i]);
}
```

2D Array (Matrix)

Declaration

```
int[][] matrix = new int[3][3];
```

Initialization

```
int[][] matrix = {
   {1,2,3},
   {4,5,6},
   {7,8,9} };
```

Access

```
System.out.println(matrix[1][2]);  Output:
```

6

Traversing 2D Array

```
for(int i = 0; i < matrix.length; i++) {
   for(int j = 0; j < matrix[i].length; j++) {
      System.out.print(matrix[i][j] + " ");
   }
   System.out.println();
}
```

Common Exceptions:

ArrayIndexOutOfBoundsException

Occurs when index exceeds size.

Example: int[] arr = {1,2,3};
System.out.println(arr[5]);

Important points:

- **Size of an array is fixed once created.**
- **length (no brackets) is used for arrays, but length() is used for Strings.**
- **Wrong index gives ArrayIndexOutOfBoundsException.**
- **Default values: int = 0, double = 0.0, boolean = false, String = null.**

length in Arrays :

What is length?

length tells you how many elements an array can hold. In Java, length is a property (field) of the array, not a method. So write it without brackets.

int[] marks = {70, 85, 90, 65, 88};

System.out.println(marks.length); // 5

Correct: marks.length
Wrong: marks.length() (compile error)

Last index = length - 1

Index starts at 0, so the last valid index is always length - 1.

int[] a = {10, 20, 30, 40};

System.out.println(a.length); // 4

System.out.println(a[a.length - 1]); // 40 (last element)

Using length in loops :

This is the most common use. You don't need to hard-code the size.

int[] nums = {5, 10, 15, 20};

for (int i = 0; i < nums.length; i++) {

System.out.println(nums[i]); }

The Arrays class :

It is in the package java.util, so write import **java.util.Arrays**;. It has ready-made methods, so you don't need to write loops.

Method	Use
Arrays.toString(arr)	Print a 1D array
Arrays.deepToString(arr)	Print a 2D array
Arrays.sort(arr)	Sort in ascending order
Arrays.binarySearch(arr, key)	Search (array **must be sorted first**)
Arrays.fill(arr, value)	Fill all positions with one value
Arrays.copyOf(arr, newLength)	Copy into a new array
Arrays.equals(a, b)	Check if two arrays have the same content

- An important point is that **Arrays.sort() modifies the original array**.
- It does not just display a sorted version; the elements of the original array are rearranged.

Sorting an Array — Arrays.sort()

The Arrays.sort() method is used to sort the elements of an array in ascending order.

```
import java.util.Arrays;
class Main {
   public static void main(String[] args) {
      int[] arr = {50, 20, 40, 10, 30};
      Arrays.sort(arr);
      System.out.println(Arrays.toString(arr));
   } }
```

Searching an Array — Arrays.binarySearch()

```
import java.util.Arrays;

class Main {

  public static void main(String[] args) {

    int[] arr = {50, 20, 40, 10, 30};

    Arrays.sort(arr);

    int result = Arrays.binarySearch(arr, 30);

    System.out.println("Array: " + Arrays.toString(arr));

    System.out.println("Index: " + result);

  } }
```

Strings :

A **String** is a sequence of characters. In Java, String is a **class**, not a primitive type.

String s1 = "Hello"; // string literal

String s2 = new String("Hello"); // using new

Strings are immutable :

Immutable means that once a String is created, it **cannot be changed**. Any change creates a **new** String.

String s = "Java";

s.concat(" Programming");

System.out.println(s); // Java (not changed!)

s = s.concat(" Programming"); // new string assigned back

System.out.println(s); // Java Programming

String pool, == and equals()

```
String a = "Hi";
String b = "Hi";
String c = new String("Hi");
System.out.println(a == b);       // true  (same object in pool)
System.out.println(a == c);       // false (different object)
System.out.println(a.equals(c));  // true  (same content)
```

- == compares **reference** (memory address).
- equals() compares **content**. Always use this for Strings.

Common String methods

```
String s = "Hello World";
s.length()            // 11
s.charAt(1)           // 'e'
s.indexOf('o')        // 4 (first position)
s.lastIndexOf('o')    // 7
s.substring(6)        // "World"
s.substring(0, 5)     // "Hello" (end index not included)
s.toUpperCase()       // "HELLO WORLD"
s.toLowerCase()       // "hello world"
s.concat("!")         // "Hello World!"
s.replace('l', 'L')   // "HeLLo WorLd"
s.contains("World")   // true
```

```
s.startsWith("He")      // true
s.endsWith("d")         // true
s.isEmpty()             // false
s.equals("hello world")          // false
s.equalsIgnoreCase("hello world") // true
```

Split a string:

```
String line = "red,green,blue";
String[] parts = line.split(",");
for (String p : parts) {
   System.out.println(p);
}
```

StringBuffer and StringBuilder

Because String is immutable, joining or changing it again and again creates many extra objects, which is slow. **StringBuffer** and **StringBuilder** are **mutable**: the same object can be changed.

Common methods (same in both)

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");       // Hello World
sb.insert(0, "Hey! ");     // Hey! Hello World
sb.replace(0, 4, "Oh");    // Oh Hello World
sb.delete(0, 3);           // Hello World
sb.reverse();              // dlroW olleH
sb.length();               // 11
```

```
sb.charAt(0);            // 'd'
sb.setCharAt(0, 'D');    // change one character
sb.toString();           // convert back to String
```

StringBuffer works exactly the same way:

```
StringBuffer sbf = new StringBuffer("Java");
sbf.append(" Notes");
System.out.println(sbf);    // Java Notes
```

Difference

Point	String	StringBuffer	StringBuilder
Mutable?	No (immutable)	Yes	Yes
Thread-safe?	Yes (immutable)	Yes (synchronized)	No
Speed	Slow for many changes	Slower than StringBuilder	**Fastest**
Use when	Value does not change	Many threads use same object	Single thread (most common case)

---------- ooo ----------